

A Multi-Agent Self-Supervised State Representation Framework for Automated Algorithm Configuration

Xiaotong Liu, Ye Tian, *Senior Member, IEEE*, Shangshang Yang, Zimo Sheng, Xingyi Zhang, *Fellow, IEEE*

Abstract—Automated algorithm configuration is an emerging area in evolutionary algorithms (EAs), where reinforcement learning (RL) has been increasingly adopted. However, most existing RL-assisted EAs still rely on simple handcrafted statistical information to construct states, which is often problem-dependent and may fail to capture complex search dynamics. To address this issue, this paper proposes a multi-agent self-supervised state representation framework with auxiliary guidance for providing agent-specific states to policy networks in RL-assisted EAs. Specifically, population observations collected online during the early stage of evolution are used to train a state encoder equipped with a multi-head readout. The encoder learns shared population tokens from these observations, while the multi-head readout derives agent-specific states, and both components are trained by integrating contrastive self-supervision with auxiliary guidance. Once trained, the encoder and readout are deployed to generate states from current population observations for downstream policy learning. The proposed framework is instantiated in four representative RL-assisted EA testbeds and evaluated across different optimization scenarios. Comparisons with three state representation methods show that MASSR generally provides more effective states for policy learning, while ablation and transfer studies further demonstrate the effectiveness and flexibility of the proposed framework.

Index Terms—Automated algorithm configuration, multi-objective optimization, self-supervised, multi-agent reinforcement learning.

I. INTRODUCTION

In recent years, reinforcement learning (RL) assisted evolutionary algorithms (EAs) [1], [2] have emerged as an effective paradigm for automated algorithm configuration [3], [4]. By modeling key algorithmic components as sequential decision-making processes [5], RL enables adaptive control based on the current search state throughout the evolutionary search. The effectiveness of RL decision making depends heavily on state input design [6], which typically relies on performance indicators and statistical features derived from population observations, including objective values and decision variables. However, directly using handcrafted states as policy inputs may restrict the representation capacity of RL-assisted EAs and fail to fully characterize complex population evolution processes [7]. In contrast, a neural-network-based state encoder provides a data-driven way to learn latent representations

from population observations, making it possible to capture implicit information beyond handcrafted state features [8]–[10]. This highlights the importance of learning a state encoder that can generate states online from population observations to support RL decision making during evolution.

Neural-network-based state encoders map population observations into compact latent state representations that can be used as inputs for policy learning [11]. Existing studies can be broadly categorized into three learning paradigms, as illustrated in Fig. 1. The first category is unsupervised representation learning, where representations are typically learned from population observations through dimensionality reduction or autoencoder-based encoding. In this paradigm, the encoder compresses the input into a compact latent vector and the decoder reconstructs the original observations, with reconstruction loss used as the training signal, as in CMODRL [12], MO-ELA [13], and DoE2Vec [14]. The second category is self-supervised representation learning, where the encoder is trained using self-generated supervisory signals prior to or separately from downstream policy learning, and the trained encoder is then used to provide state inputs for policy learning, as in Deep-ELA [15] and CURL [16]. The third category is task-driven representation learning, where the representation model is trained with signals derived from algorithm configuration and decision-making tasks, as in NeurELA [6] and RLDE-AFL [7].

However, with the increasing adoption of multi-agent reinforcement learning (MARL) in RL-assisted EAs to support more complex and agent-specific algorithm configuration [17], the above paradigms remain inadequate for multi-agent settings [18], since they still mainly produce a unified state representation from population observations. Unsupervised representation learning compresses population observations into a unified latent state through dimensionality reduction or reconstruction-based encoding, inevitably causing information loss when multiple agents require different information from the same population [19]. Self-supervised representation learning trains the encoder with self-generated pretext signals rather than reconstruction objectives, improving representation robustness and training independence. However, it still lacks explicit guidance from algorithm configuration decisions. Task-driven representation learning enhances task relevance using signals from algorithm configuration and decision-making tasks, but existing methods still mainly provide unified representations rather than differentiated state inputs for heterogeneous agents in MARL.

To overcome these limitations, this paper proposes a multi-agent self-supervised state representation framework with auxiliary guidance (MASSR). Different from existing unified-

Manuscript received –. This work was supported in part by the National Natural Science Foundation of China (No. 62522301, No. 62276001, No. W2441019) and the Anhui Provincial Natural Science Foundation (No. 2308085J03). (Corresponding author: Ye Tian.)

The authors are with the Key Laboratory of Intelligent Computing and Signal Processing of Ministry of Education, School of Computer Science and Technology, Anhui University, Hefei 230601, China. (email: xtongliu25@gmail.com; field910921@gmail.com; yangshang0308@gmail.com; shengzimo@ahu.edu.cn; xyzhanghust@gmail.com).

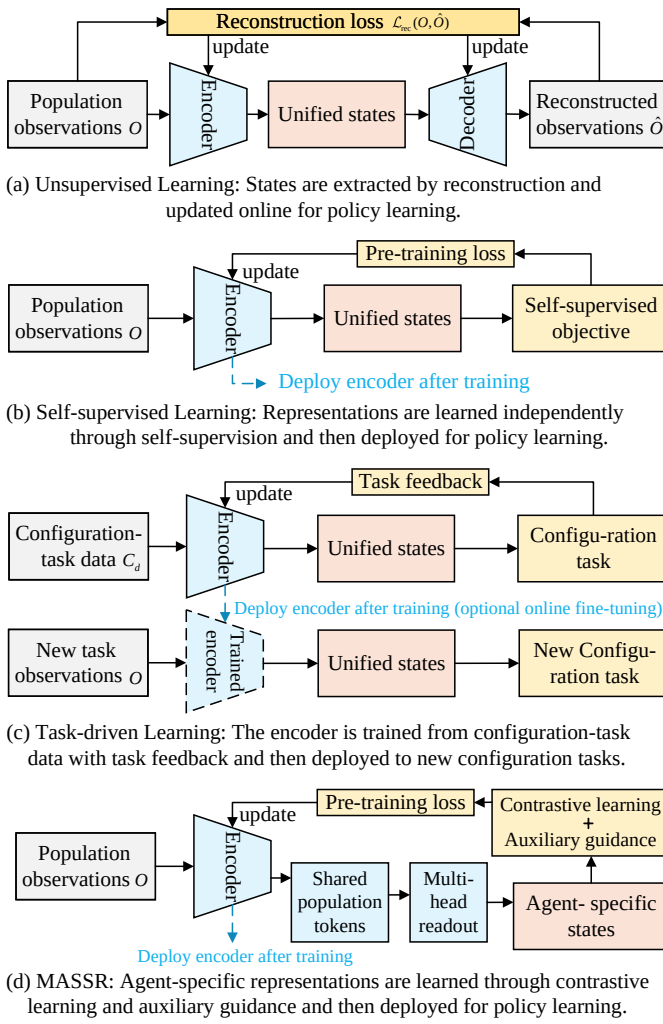


Fig. 1. Conceptual comparison of four state representation paradigms for policy learning. The figure highlights their differences in training signals, coupling with policy learning, and the form of generated state representations.

state paradigms, MASSR first learns shared population tokens from population observations and then derives agent-specific states through a multi-head readout mechanism, thereby reducing information loss while providing differentiated state inputs for agents with different decision roles. Furthermore, self-supervised learning enables independent encoder training but may lack explicit guidance for algorithm configuration decisions, while task-driven learning improves task relevance but still rarely considers heterogeneous state requirements in MARL. As illustrated in Fig. 1, MASSR integrates contrastive self-supervision with auxiliary guidance to learn stable and configuration-relevant representations for agent-specific decision-making.

The main contributions of this work are as follows:

- 1) This work proposes a multi-agent self-supervised state representation framework with auxiliary guidance, termed MASSR. Different from existing unified-state schemes, MASSR first learns shared population tokens from population observations and then derives agent-specific states through a multi-head readout mechanism, enabling multiple agents to obtain differentiated de-

cision information from the same population. In addition, contrastive self-supervision is integrated with auxiliary guidance to improve the quality of the learned representations and enhance their discriminability for downstream policy learning.

- 2) Based on MASSR, this work further develops a modular state representation paradigm for multi-agent reinforcement learning-assisted algorithm configuration (MARL-AC). In this paradigm, a state representation module is first learned through representation learning with auxiliary guidance and then deployed in MARL-AC methods to map current population observations into agent-specific states for downstream policy learning. As illustrated in Fig. 1, compared with unsupervised, self-supervised, and task-driven representation paradigms, the proposed paradigm combines independent representation learning with configuration-related auxiliary guidance, while providing differentiated state representations for heterogeneous decision roles.
- 3) The effectiveness and generality of MASSR are validated by instantiating it into four representative RL-assisted EA testbeds across different optimization scenarios. Comprehensive comparisons with handcrafted, DAE-based, and NeurELA-based state representations show that MASSR generally provides more effective policy states and competitive optimization performance. In addition, comparisons with existing RL-assisted EAs, ablation studies, head-wise representation analyses, and transfer experiments further verify the effectiveness, role-specific representation ability, and flexible applicability of the proposed framework.

The rest of this paper is organized as follows. Section II reviews state construction methods in RL-assisted EAs and discusses the motivation of this work. Section III presents the proposed MASSR framework, including representation learning with auxiliary guidance and its deployment in MARL-AC. Section IV reports the empirical studies across different RL-assisted EA testbeds, including state representation comparisons, comparisons with existing RL-assisted EAs, ablation studies, head-wise analyses, and transfer experiments. Finally, Section V concludes this paper and outlines future research directions.

II. RELATED WORK

A. Model of Algorithm Configuration

Let A denote a configurable EA, and let $\Theta = \Theta_1 \times \Theta_2 \times \dots \times \Theta_K$ denote its configuration space, where each Θ_k corresponds to one configurable item of the EA, which may be either a numerical parameter or a selectable algorithmic component [20]. Therefore, a configuration $\theta \in \Theta$ represents a complete assignment of all configurable components and parameters [21], [22]. In classical algorithm configuration, the goal is to find a fixed configuration that achieves the best expected performance over a problem distribution D , which can be written as $\theta^* \in \arg \min_{\theta \in \Theta} \mathbb{E}_{i \sim D} [c(\theta, i)]$, where i denotes a problem instance and $c(\theta, i)$ denotes the cost of applying configuration θ to solve i . However, for RL-assisted

EAs, the appropriate configuration may vary across different evolutionary stages. Therefore, instead of optimizing a single fixed configuration, it is more suitable to optimize a dynamic configuration policy π , and the problem can be formulated as

$$\pi^* \in \arg \min_{\pi \in \Pi} \mathbb{E}_{i \sim D} [c(\pi, i)], \quad (1)$$

where Π denotes the policy space and $c(\pi, i)$ denotes the overall optimization cost obtained by controlling the EA on instance i with policy π throughout the whole evolutionary process.

To solve the above problem by reinforcement learning, the online configuration process is modeled as a sequential decision-making problem. At generation t , the RL agent receives a state s_t , which is constructed as $s_t = \phi(P_t, H_t)$, where P_t denotes the current population, H_t denotes relevant historical information, and $\phi(\cdot)$ denotes the state construction function. Based on s_t , the agent outputs an action $a_t = \pi(s_t)$, where $a_t \in \Theta$ specifies the configuration adopted at the current generation, including both component choices and parameter settings. After that, the EA evolves to the next population according to $P_{t+1} = \mathcal{T}(P_t, a_t, \xi_t)$, where $\mathcal{T}(\cdot)$ denotes one-step evolutionary transformation and ξ_t denotes stochastic variation. The reward r_t is used to measure the performance change caused by the current configuration decision. If a performance indicator $\mathcal{M}(\cdot)$ is to be maximized, the reward can be defined as $r_t = \mathcal{M}(P_{t+1}) - \mathcal{M}(P_t)$; if a loss indicator $\mathcal{L}(\cdot)$ is to be minimized, it can be defined as $r_t = \mathcal{L}(P_t) - \mathcal{L}(P_{t+1})$ [23], [24]. The objective of RL is then to learn a policy that maximizes the expected cumulative reward over the evolutionary process.

B. State Representation in RL-assisted EAs

In RL-assisted EAs, the state serves as the key interface between the optimization process and the RL policy, and its construction directly affects the effectiveness of policy learning [25]. Early studies mostly relied on handcrafted state design [26], where the current search status is described by manually selected features derived from prior knowledge and optimization indicators, such as population diversity [27], convergence, feasibility [28] or constraint-violation information [29], and evolutionary progress [30]. However, such designs depend heavily on human expertise and task-specific feature engineering [31], [32], which motivates recent efforts to learn state representations automatically from population observations. Researchers have started to learn state representations from population observations through explicit representation learning objectives, which can be generally grouped into unsupervised, self-supervised, and task-driven representation learning. Formally, learning-based state representation aims to learn an encoder $f_\theta(\cdot)$ that maps a raw population observation O into a latent state s , i.e., $s = f_\theta(O)$, where O may denote the population objective matrix, decision-variable matrix, or other observation forms collected during evolution. Rather than directly feeding O into policy learning, the encoder is optimized under explicit objectives so that the resulting state s can retain information that is more useful for downstream decision-making. In general, this process can be formulated

as $\min_\theta \mathcal{L}_{\text{rep}}(\theta; O)$, where $\mathcal{L}_{\text{rep}}$ may be defined by reconstruction objectives, self-generated pretext signals, task-driven objectives, or auxiliary guidance information.

Some studies introduced unsupervised learning to compress raw population observations into low-dimensional representations, providing a more compact description of the current search status. CModRL [12] employs a denoising autoencoder (DAE) to map each solution's objective vector into a compressed scalar representation, thereby transforming the population into a lower-dimensional state vector for deep RL. MO-ELA [13] extracts key landscape features from extensive sampled decision-space and objective-space data, yielding a more informative representation of multi-objective optimization problems for algorithm selection. DoE2Vec [14] adopts a variational autoencoder to compress sampled optimization data into compact latent representations for automatic landscape characterization and algorithm selection. Recurrent Capsule Networks [33] derive informative latent states from high-dimensional observations through future-observation prediction, providing automatically extracted representations for downstream decision-making. However, unsupervised representation learning is mainly driven by reconstruction or prediction objectives. Therefore, the learned latent state may preserve information that is sufficient for recovering population observations, while failing to retain the information most useful for downstream RL policy learning. This limitation is particularly evident in MARL settings, where different agents may require different views of the same population. Compressing such observations into a single latent state makes it difficult to preserve the differentiated information required by different agents [19].

To alleviate the limitation of reconstruction- or prediction-oriented objectives in unsupervised learning, some studies turned to self-supervised representation learning. In this paradigm, supervisory signals are generated from population observations through predefined pretext tasks, encouraging the encoder to learn more robust and discriminative representations beyond simple observation reconstruction. Seiler *et al.* [15] proposed Deep-ELA, which learns transformer-based landscape representations through self-supervised pretraining on sampled decision- and objective-space data before being applied to downstream optimization tasks. Laskin *et al.* [16] proposed CURL, which learns informative state representations from raw pixel observations through contrastive representation learning, enabling reinforcement learning to operate directly on high-dimensional visual inputs. Nevertheless, these methods mainly construct training signals from observation-level transformations or sampling variations, which encourages representation consistency but does not explicitly indicate which population characteristics are critical for algorithm configuration decisions. Therefore, the learned representations may still be insufficiently aligned with downstream decision requirements.

To better profile optimization status for algorithm configuration across different algorithms and problem instances, task-driven representation learning trains representation models under configuration-oriented objectives. Ma *et al.* [6] proposed NeurELA, a representative and advanced offline-

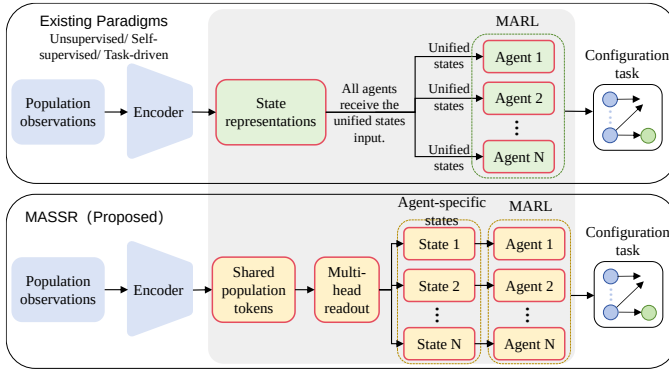


Fig. 2. Deployment-level comparison of state representation in MARL. Existing methods usually provide a unified state input to all agents, whereas MASSR generates agent-specific states from shared population tokens through a multi-head readout mechanism, enabling differentiated decision information for heterogeneous agents.

trained neural state encoder for MetaBBO, which learns a two-stage attention-based neural landscape analyzer through multi-task pretraining over different MetaBBO algorithms and problem instances [34]–[36]. Guo *et al.* [7] proposed RLDE-AFL, which incorporates a NeurELA-based feature extractor into an RL-based differential evolution framework to support automated operator selection and parameter control. Wu *et al.* [37] proposed SEEMOO, where an attention-based state extractor is used to unify population convergence, performance metrics, and multi-objective information, and a DQN-based selector dynamically schedules surrogate models according to the current optimization state. de Bruin *et al.* [38] investigated the integration of state representation learning objectives into deep RL, where auxiliary representation losses regularize the learned state while the RL objective also helps shape the representation. Liu *et al.* [39] studied state representation modeling in a deep reinforcement learning recommendation framework, where user–item interactions are explicitly modeled to construct state representations for Actor–Critic policy learning.

C. Motivation

Although learning-based state representation has reduced the reliance on handcrafted state design, existing paradigms typically produce a unified state for downstream policy learning. As shown in Fig. 2, this unified representation is shared across all agents in MARL-assisted EAs, which may limit its effectiveness when agents have heterogeneous decision roles. Specifically, unsupervised learning usually compresses population observations into a unified latent state through reconstruction- or prediction-based objectives, but such objectives are not directly aligned with downstream configuration decisions. Self-supervised learning trains the encoder with self-generated supervisory signals before downstream policy learning, but these signals are usually constructed from the data itself and may not explicitly indicate which population characteristics are critical for algorithm configuration decisions. Task-driven representation learning improves the task relevance of learned states by using feedback derived from

configuration tasks, but existing methods still mainly provide unified representations rather than differentiated state inputs for heterogeneous agents.

To overcome these limitations, we propose a multi-agent self-supervised state representation framework (MASSR). Different from existing unified-state paradigms, MASSR learns shared population tokens from population observations and derives agent-specific states through a multi-head readout mechanism. (1) Compared with unsupervised learning, MASSR avoids directly compressing all population information into a single unified state, while preserving shared population information and allowing different agents to focus on different decision-relevant aspects. (2) Compared with self-supervised learning, MASSR complements contrastive self-supervision with auxiliary guidance targets, so that representation learning is informed not only by data-derived supervisory signals but also by optimization-related population information for configuration decisions. (3) Compared with task-driven representation learning, MASSR not only preserves task relevance through auxiliary guidance, but also associates different readout heads with head-specific guidance targets during representation learning, enabling the multi-head readout to derive agent-specific states from shared population tokens for heterogeneous agents.

III. THE PROPOSED MASSR FRAMEWORK

The proposed MASSR framework follows a two-stage architecture, namely representation learning and policy learning, as illustrated in Fig. 3. The key idea is to first learn a deployable state representation module from population observations by integrating contrastive self-supervision with auxiliary guidance, and then deploy the learned module in a downstream MARL-AC framework for policy learning. In Stage I, a shared population encoder is used to extract shared population tokens from population observations, and a multi-head readout module further transforms them into agent-specific states. In Stage II, the trained encoder and readout are deployed as the learned state representation module, which provides specialized state inputs for downstream multi-agent decision making.

A. Self-Supervised State Representation Learning with Auxiliary Guidance

In Stage I, the shared population encoder and the multi-head readout module are jointly trained through head-specific contrastive self-supervision and auxiliary guidance learning, as illustrated in Fig. 4. The overall training procedure is summarized in Algorithm 1. Specifically, the shared encoder maps each population observation into shared population tokens, and the multi-head readout further transforms these tokens into agent-specific states. Through this representation learning process, the learned representation module is encouraged to produce agent-specific states that are robust to observation perturbations while preserving search-relevant information for downstream decision making.

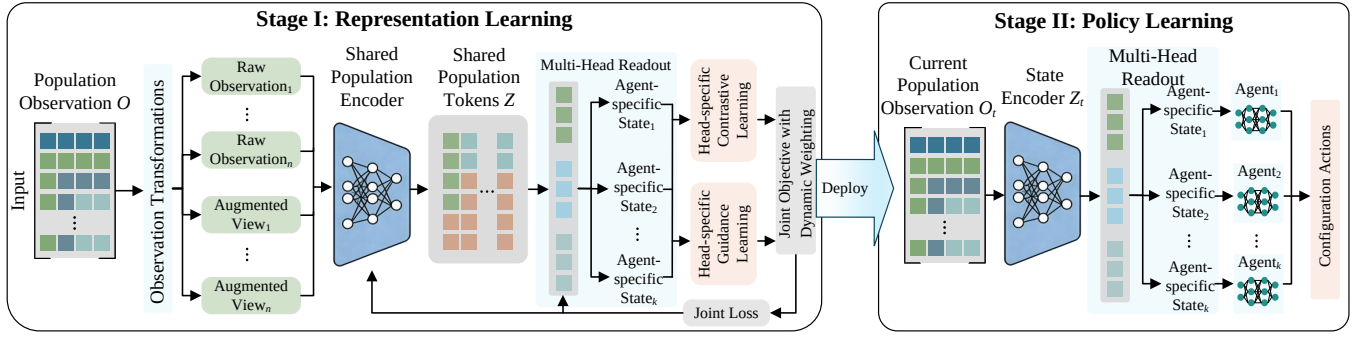


Fig. 3. Overview of the proposed MASSR framework. In Stage I, population observations are transformed into multiple views, encoded into shared population tokens, and further mapped to agent-specific states by a multi-head readout module. In Stage II, the learned representation module is deployed in MARL-AC, where current population observations are converted into agent-specific states for downstream policy learning and configuration action generation.

At the beginning of each independent optimization run, population observations are collected during an early random-configuration phase using the first 20% of the maximum function evaluations (MaxFEs). During this phase, the controllable components of the EA are configured using randomly selected actions, and the resulting population observations are recorded as training samples. These evaluations are included in the total MaxFEs. Once 20% of MaxFEs has been reached, the encoder and multi-head readout are trained once using the observations collected in the current run, without additional objective-function evaluations. The evolutionary search then continues directly from the current population and function-evaluation counter, with the remaining 80% of MaxFEs used for Stage-II policy learning and optimization.

Let the current population contain N individuals, where each individual is represented by an observation vector constructed from selected individual-level features. These features may include the M objective values, the D decision variables, or their combination. In the proposed framework, the population observation is constructed from both decision variables and objective values, with the feature dimension given by $C = D + M$. The decision variables are normalized to $[0, 1]$ according to their lower and upper bounds, while the objective values are normalized by dimension-wise min-max normalization within the current population. The normalized population observation is organized as

$$\mathbf{O} = [\mathbf{o}_1, \mathbf{o}_2, \dots, \mathbf{o}_N]^\top \in \mathbb{R}^{N \times C}, \quad (2)$$

where $\mathbf{o}_i \in \mathbb{R}^C$ denotes the normalized observation of the i -th individual, and C denotes the corresponding feature dimension. To enable representation learning, two transformed views, namely $\mathbf{O}^{(1)} = \mathcal{T}_1(\mathbf{O})$ and $\mathbf{O}^{(2)} = \mathcal{T}_2(\mathbf{O})$, are generated from the original population observation through stochastic observation transformations, where $\mathcal{T}_1(\cdot)$ and $\mathcal{T}_2(\cdot)$ denote two random transformation operators. The stochastic transformations include population subsampling, additive perturbation, and feature masking [40], [41], and their implementation settings are provided in Section SI of the Supplementary Materials. In addition, an auxiliary guidance vector is extracted from the original population observation by a fixed, non-trainable guidance extractor, and head-specific binary masks are used to select the guidance components associated with

different readout heads during Stage-I representation learning.

Given a population observation $\mathbf{O}^* = [\mathbf{o}_1^*, \mathbf{o}_2^*, \dots, \mathbf{o}_{N^*}^*]^\top \in \mathbb{R}^{N^* \times C}$, where $\mathbf{O}^* \in \{\mathbf{O}, \mathbf{O}^{(1)}, \mathbf{O}^{(2)}\}$, the shared population encoder maps the input observation into structured population tokens, as summarized in Algorithm 2. Specifically, each individual observation is first projected into a latent embedding space through an individual embedding network, yielding $\mathbf{H}^* = \phi(\mathbf{O}^*)$, where $\mathbf{H}^* \in \mathbb{R}^{N^* \times d_h}$ denotes the latent embedding matrix and d_h is the embedding dimension. An attention scoring network is then applied to estimate the relative importance of individual embeddings, producing $\mathbf{e}^* = \chi(\mathbf{H}^*)$, where $\mathbf{e}^* \in \mathbb{R}^{N^* \times 1}$ denotes the attention score vector. The normalized attention weights are obtained by applying the softmax operation over the population dimension:

$$\omega_i^* = \frac{\exp(e_i^*)}{\sum_{j=1}^{N^*} \exp(e_j^*)}, \quad (3)$$

where $\omega^* = [\omega_1^*, \dots, \omega_{N^*}^*]^\top \in \mathbb{R}^{N^* \times 1}$. The latent embeddings are then reweighted in a row-wise manner to form an intermediate token representation, with $\mathbf{Z}_h^* = \omega^* \odot \mathbf{H}^*$, where $\odot$ denotes row-wise multiplication.

To obtain a fixed-size token representation, the encoder constructs N_z representative tokens from the N^* reweighted individual embeddings. Specifically, for the i -th individual, a population-distance score is computed as the sum of its distances to all other individuals in the normalized observation space:

$$r_i^* = \sum_{j=1}^{N^*} \|\mathbf{o}_i^* - \mathbf{o}_j^*\|_2, \quad \mathcal{I}_z^* = \arg \text{top}_{N_z} \left(\{r_i^*\}_{i=1}^{N^*} \right), \quad (4)$$

where r_i^* denotes the population-distance score of the i -th individual, and $\arg \text{top}_{N_z}(\cdot)$ returns the indices of the N_z largest scores. The selected reweighted embeddings are then transformed by the token construction and projection module $\psi(\cdot)$, producing

$$\mathbf{Z}^* = \psi(\mathbf{Z}_h^*[\mathcal{I}_z^*, :]), \quad \mathbf{Z}^* \in \mathbb{R}^{N_z \times d_z}, \quad (5)$$

where $\mathbf{Z}^*$ denotes the shared population tokens, N_z is the fixed number of tokens, and d_z is the token dimension.

Based on the shared population tokens, a multi-head readout module is further introduced to generate agent-specific states

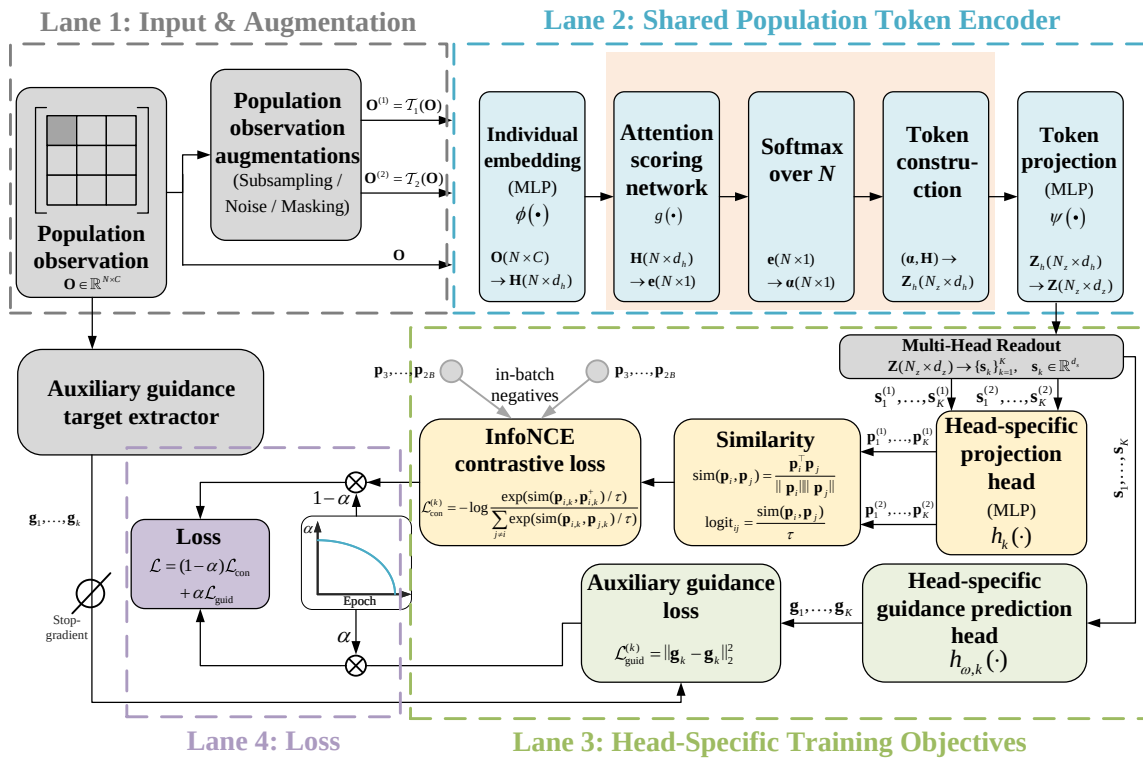


Fig. 4. Workflow of Stage I in the proposed MASSR framework. Population observations are transformed into multiple views, encoded into shared population tokens, and further mapped to agent-specific states by the multi-head readout. The representation module is jointly optimized by head-specific contrastive self-supervision and auxiliary guidance learning.

for subsequent representation-learning objectives. Specifically, all readout heads take the same shared token representation as input, but each head is associated with its own parameter set and extracts a decision-oriented state representation for one agent. Given $\mathbf{Z}^* \in \{\mathbf{Z}, \mathbf{Z}^{(1)}, \mathbf{Z}^{(2)}\}$, the k -th readout head first aggregates the shared tokens through a head-specific attention pooling operation and then maps the pooled representation into a fixed-dimensional state,

$$\mathbf{s}_k^* = f_k(\mathbf{Z}^*; \Theta_{\text{read}}^{(k)}) = \rho_k \left(\sum_{j=1}^{N_z} \beta_{k,j}^* \mathbf{z}_j^* \right), \quad (6)$$

where $\beta_{k,j}^*$ denotes the attention weight assigned by the k -th head to the j -th token. It is computed as

$$\beta_{k,j}^* = \frac{\exp((\mathbf{q}_k)^\top \mathbf{z}_j^*)}{\sum_{\ell=1}^{N_z} \exp((\mathbf{q}_k)^\top \mathbf{z}_\ell^*)}, \quad (7)$$

where $\mathbf{q}_k$ is the head-specific query vector and $\rho_k(\cdot)$ is the state projection function. Here, $k = 1, 2, \dots, K$, $f_k(\cdot)$ denotes the k -th readout function, $\Theta_{\text{read}}^{(k)}$ is the corresponding parameter set, and $\mathbf{s}_k^* \in \mathbb{R}^{d_s}$ is the resulting head-specific state representation. In this way, different readout heads can extract different state views from the same shared token space. Accordingly, for the original observation and its two transformed views, the multi-head readout module produces:

$$\begin{aligned} \mathbf{s}_k &= f_k(\mathbf{Z}; \Theta_{\text{read}}^{(k)}) \\ \mathbf{s}_k^{(1)} &= f_k(\mathbf{Z}^{(1)}; \Theta_{\text{read}}^{(k)}) \\ \mathbf{s}_k^{(2)} &= f_k(\mathbf{Z}^{(2)}; \Theta_{\text{read}}^{(k)}), \end{aligned} \quad (8)$$

for $k = 1, 2, \dots, K$. In this way, the shared token space is transformed into multiple decision-oriented state views, which are then used to construct the head-specific contrastive and auxiliary guidance objectives.

To jointly optimize the shared population encoder and the multi-head readout module, the learned head-specific states are further used to construct two head-specific training objectives, namely contrastive self-supervision and auxiliary guidance learning. For the contrastive branch, the head-specific states extracted from the two transformed views are first passed through the corresponding projection heads to obtain

$$\mathbf{p}_k^{(1)} = h_k(\mathbf{s}_k^{(1)}), \mathbf{p}_k^{(2)} = h_k(\mathbf{s}_k^{(2)}), k = 1, 2, \dots, K, \quad (9)$$

where $h_k(\cdot)$ denotes the projection network associated with the k -th head. The similarity between projected representations is measured by cosine similarity:

$$\text{sim}(\mathbf{p}_k^{(1)}, \mathbf{p}_k^{(2)}) = \frac{\left(\mathbf{p}_k^{(1)}\right)^\top \mathbf{p}_k^{(2)}}{\left\|\mathbf{p}_k^{(1)}\right\| \left\|\mathbf{p}_k^{(2)}\right\|}. \quad (10)$$

Based on the projected features in a mini-batch, the contrastive objective for the k -th head is defined using the InfoNCE loss, where the two transformed views derived from the same population observation form a positive pair, and the projected representations of other observations in the same mini-batch serve as in-batch negatives. For notational convenience, let $\{\mathbf{p}_{i,k}\}_{i=1}^{2B}$ denote the set of projected representations associated with the k -th head in a mini-batch, where $\mathbf{p}_{i,k}^+$ represents the positive counterpart of $\mathbf{p}_{i,k}$ derived from

Algorithm 1 MASSR representation learning

Input: $\mathcal{B}$ (mini-batches of population observations and associated search records collected during the early random-configuration phase of the current independent run), $\mathcal{T}_1, \mathcal{T}_2$ (stochastic observation transformations), $\mathcal{G}$ (non-trainable guidance extractor), $\{\mathbf{m}_k\}_{k=1}^K$ (head-specific guidance masks), $\alpha(\cdot)$ (dynamic weight schedule), T (number of training epochs), η (learning rate)

Output: trained encoder and readout parameters $(\theta, \Theta_{\text{read}})$

- 1: **Initialize:** $\theta, \Theta_{\text{read}}, \Theta_{\text{proj}}$, and Θ_{guid}
- 2: **for** $t = 1$ to T **do**
- 3: **for each** mini-batch $\{(\mathbf{O}_i, \mathcal{H}_i)\}_{i=1}^B \in \mathcal{B}$ **do**
- 4: *// Generate transformed views and encode population observations*
- 5: **for** $i = 1$ to B **do**
- 6: $\mathbf{O}_i^{(1)} \leftarrow \mathcal{T}_1(\mathbf{O}_i), \mathbf{O}_i^{(2)} \leftarrow \mathcal{T}_2(\mathbf{O}_i)$
- 7: $\mathbf{Z}_i \leftarrow \text{ENCODEPOPULATION}(\mathbf{O}_i; \theta)$
- 8: $\mathbf{Z}_i^{(1)} \leftarrow \text{ENCODEPOPULATION}(\mathbf{O}_i^{(1)}; \theta)$
- 9: $\mathbf{Z}_i^{(2)} \leftarrow \text{ENCODEPOPULATION}(\mathbf{O}_i^{(2)}; \theta)$
- 10: **end for**
- 11: *// Transform shared tokens into head-specific states*
- 12: **for** $i = 1$ to B **do**
- 13: **for** $k = 1$ to K **do**
- 14: $\mathbf{s}_{i,k} \leftarrow \text{READOUT}_k(\mathbf{Z}_i; \Theta_{\text{read}}^{(k)})$
- 15: $\mathbf{s}_{i,k}^{(1)} \leftarrow \text{READOUT}_k(\mathbf{Z}_i^{(1)}; \Theta_{\text{read}}^{(k)})$
- 16: $\mathbf{s}_{i,k}^{(2)} \leftarrow \text{READOUT}_k(\mathbf{Z}_i^{(2)}; \Theta_{\text{read}}^{(k)})$
- 17: **end for**
- 18: **end for**
- 19: *// Compute head-specific losses*
- 20: $\{\mathcal{L}_{\text{con}}^{(k)}\}_{k=1}^K \leftarrow \text{CONTRASTIVELOSS}(\{\mathbf{s}_{i,k}^{(1)}, \mathbf{s}_{i,k}^{(2)}\}_{i=1, k=1}^{B, K}, \Theta_{\text{proj}})$
- 21: $\{\mathcal{L}_{\text{guid}}^{(k)}\}_{k=1}^K \leftarrow \text{GUIDANCELOSS}(\{(\mathbf{O}_i, \mathcal{H}_i), \mathbf{s}_{i,k}\}_{i,k}, \mathcal{G}, \{\mathbf{m}_k\}_{k=1}^K, \Theta_{\text{guid}})$
- 22: *// Form the joint loss and update parameters*
- 23: **for** $k = 1$ to K **do**
- 24: $\mathcal{L}^{(k)} \leftarrow (1 - \alpha(t))\mathcal{L}_{\text{con}}^{(k)} + \alpha(t)\mathcal{L}_{\text{guid}}^{(k)}$
- 25: **end for**
- 26: $\mathcal{L} \leftarrow \sum_{k=1}^K \mathcal{L}^{(k)}$
- 27: $(\theta, \Theta_{\text{read}}, \Theta_{\text{proj}}, \Theta_{\text{guid}}) \leftarrow \text{UPDATE}(\theta, \Theta_{\text{read}}, \Theta_{\text{proj}}, \Theta_{\text{guid}}, \mathcal{L}, \eta)$
- 28: **end for**
- 29: **end for**
- 30: **end for**
- 31: **return** $(\theta, \Theta_{\text{read}})$

the same original population observation. The contrastive loss for $\mathbf{p}_{i,k}$ is defined as

$$\ell_{i,k} = -\log \frac{\exp(\text{sim}(\mathbf{p}_{i,k}, \mathbf{p}_{i,k}^+)/\tau)}{\sum_{j=1}^{2B} \mathbb{I}_{[j \neq i]} \exp(\text{sim}(\mathbf{p}_{i,k}, \mathbf{p}_{j,k})/\tau)}, \quad (11)$$

where τ is a temperature parameter, B is the mini-batch size, and $\mathbb{I}_{[\cdot]}$ is an indicator function. The contrastive loss of the k -th head is then computed as $\mathcal{L}_{\text{con}}^{(k)} = \frac{1}{2B} \sum_{i=1}^{2B} \ell_{i,k}$.

The auxiliary guidance vector is generated by a fixed, non-trainable mapping from population observations to population-

Algorithm 2 ENCODEPOPULATION

Input: population observation $\mathbf{O}^* \in \mathbb{R}^{N^* \times C}$, encoder parameters $\theta = (\Theta_\phi, \Theta_\chi, \Theta_\psi)$, token number N_z

Output: shared population tokens $\mathbf{Z}^* \in \mathbb{R}^{N_z \times d_z}$

- 1: *// Individual embedding*
- 2: $\mathbf{H}^* \leftarrow \phi(\mathbf{O}^*; \Theta_\phi)$
- 3: *// Attention scoring and normalization*
- 4: $\mathbf{e}^* \leftarrow \chi(\mathbf{H}^*; \Theta_\chi)$
- 5: $\boldsymbol{\omega}^* \leftarrow \text{SOFTMAX}(\mathbf{e}^*)$
- 6: *// Row-wise reweighting of latent embeddings*
- 7: $\mathbf{Z}_h^* \leftarrow \boldsymbol{\omega}^* \odot \mathbf{H}^*$
- 8: *// Representative token selection*
- 9: $\mathcal{I}_z^* \leftarrow \arg \text{top}_{N_z} \left(\left\{ \sum_{j=1}^{N^*} \|\mathbf{o}_i^* - \mathbf{o}_j^*\|_2 \right\}_{i=1}^{N^*} \right)$
- 10: *// Token construction and projection*
- 11: $\mathbf{Z}^* \leftarrow \psi(\mathbf{Z}_h^*[\mathcal{I}_z^*, :]; \Theta_\psi)$
- 12: **return** $\mathbf{Z}^*$

level optimization statistics. Specifically, it is defined as $\mathbf{g} = \mathcal{G}(\mathbf{O}, \mathcal{H}) = [g_1, g_2, \dots, g_{d_g}]^\top \in [0, 1]^{d_g}$, where each component g_r denotes a normalized statistic describing one aspect of the current search status, such as progress, improvement, diversity, distribution, stagnation, or population movement. For the k -th readout head, a binary guidance mask is defined as $\mathbf{m}_k = [m_{k,1}, m_{k,2}, \dots, m_{k,d_g}]^\top \in \{0, 1\}^{d_g}$, which selects the guidance components used to supervise the k -th readout head according to its decision role. These guidance vectors are used only in the auxiliary prediction branch during Stage-I representation learning, and the concrete indicator definitions used in this study are provided in Section SII of the Supplementary Materials.

For the auxiliary guidance branch, the head-specific states extracted from the original population observations are fed into the corresponding guidance prediction heads, yielding $\hat{\mathbf{g}}_{i,k} = h_{\omega,k}(\mathbf{s}_{i,k})$ for the i -th sample and the k -th head, where $h_{\omega,k}(\cdot)$ denotes the guidance prediction head associated with the k -th head, and $\hat{\mathbf{g}}_{i,k} \in \mathbb{R}^{d_g}$ is the predicted guidance vector. Given the guidance vector $\mathbf{g}_i = \mathcal{G}(\mathbf{O}_i, \mathcal{H}_i)$ and the head-specific mask $\mathbf{m}_k$, the auxiliary guidance loss of the k -th head is defined as

$$\mathcal{L}_{\text{guid}}^{(k)} = \frac{1}{B} \sum_{i=1}^B \frac{\|\mathbf{m}_k \odot (\hat{\mathbf{g}}_{i,k} - \mathbf{g}_i)\|_2^2}{\max(\|\mathbf{m}_k\|_0, 1)}. \quad (12)$$

Finally, for the k -th head, the joint training objective is defined as

$$\mathcal{L}^{(k)} = (1 - \alpha(t))\mathcal{L}_{\text{con}}^{(k)} + \alpha(t)\mathcal{L}_{\text{guid}}^{(k)}, \quad (13)$$

where $\alpha(t)$ is the dynamic weight assigned to the auxiliary guidance loss. In this study, $\alpha(t)$ follows an exponential decay schedule:

$$\alpha(t) = \exp\left(-\lambda_\alpha \frac{t-1}{T-1}\right), \quad t = 1, 2, \dots, T, \quad (14)$$

where T is the total number of training epochs and λ_α controls the decay rate. The contrastive weight is given by $1 - \alpha(t)$, so the training process gradually shifts from stronger

Algorithm 3 DEPLOYMASSR

Input: current population observation $\mathbf{O}_t$, trained encoder and readout parameters $(\theta, \Theta_{\text{read}})$, agent policies $\{\pi_k\}_{k=1}^K$
Output: agent-specific states $\{\mathbf{s}_{t,k}\}_{k=1}^K$ and joint action $\mathbf{a}_t$

- 1: *// Encode the current population observation into shared population tokens*
- 2: $\mathbf{Z}_t \leftarrow \text{ENCODEPOPULATION}(\mathbf{O}_t; \theta)$
- 3: *// Generate agent-specific states through the trained multi-head readout*
- 4: **for** $k = 1$ to K **do**
- 5: $\mathbf{s}_{t,k} \leftarrow \text{READOUT}_k(\mathbf{Z}_t; \Theta_{\text{read}}^{(k)})$
- 6: **end for**
- 7: *// Select actions based on the generated states*
- 8: **for** $k = 1$ to K **do**
- 9: $a_{t,k} \leftarrow \pi_k(\mathbf{s}_{t,k})$
- 10: **end for**
- 11: $\mathbf{a}_t \leftarrow [a_{t,1}, a_{t,2}, \dots, a_{t,K}]$
- 12: **return** $\{\mathbf{s}_{t,k}\}_{k=1}^K, \mathbf{a}_t$

auxiliary guidance to stronger contrastive self-supervision. The overall training objective is then obtained by aggregating the joint losses of all heads, i.e., $\mathcal{L} = \sum_{k=1}^K \mathcal{L}^{(k)}$. Since the head-specific contrastive and auxiliary guidance objectives are defined on the agent-specific states produced by the multi-head readout, the resulting training signals are back-propagated through both the readout module and the shared population encoder. Specifically, each readout head is mainly updated by its own joint loss $\mathcal{L}^{(k)}$, while the shared population encoder is jointly updated by the aggregated gradients from all heads. By jointly optimizing the contrastive and auxiliary guidance objectives, the shared population encoder and the multi-head readout module are encouraged to learn representations that are both robust to observation transformations and capable of preserving search-relevant information for heterogeneous downstream decision making.

B. Deployment in MARL-AC for Policy Learning

After Stage-I representation learning, the learned shared population encoder and multi-head readout module are frozen and deployed in MARL-AC as the state representation module for downstream policy learning, as illustrated in Stage II of Fig. 3. The evolutionary search continues from the current population and function-evaluation counter reached after the first 20% of MaxFEs, with the remaining evaluation budget used for Stage-II optimization. Algorithm 3 summarizes the deployment of the learned representation module, while Algorithm 4 outlines the subsequent optimization procedure. At each generation, the deployed module transforms the current population observation into shared population tokens and further maps them into agent-specific states, which are provided to the corresponding agents for action selection and online policy learning.

During Stage II, at generation t , let $\mathbf{O}_t \in \mathbb{R}^{N \times C}$ denote the normalized population observation constructed from the current population, where N is the population size and $C =$

Algorithm 4 Stage-II Procedure of MARL-AC with MASSR

Input: current population P reached after the first 20% of MaxFEs, current function-evaluation counter FEs, trained encoder and readout parameters $(\theta, \Theta_{\text{read}})$, current agent policies $\{\pi_k\}_{k=1}^K$, replay buffer $\mathcal{R}$, maximum function evaluations MaxFEs
Output: final population P

- 1: **while** FEs < MaxFEs **do**
- 2: Construct the current population observation $\mathbf{O}$ from P
- 3: *// Obtain agent-specific states and joint action via MASSR*
- 4: $\{\mathbf{s}_k\}_{k=1}^K, \mathbf{a} \leftarrow \text{DEPLOYMASSR}(\mathbf{O}; \theta, \Theta_{\text{read}}, \{\pi_k\}_{k=1}^K)$
- 5: *// Configure the EA and perform one search step*
- 6: Apply $\mathbf{a}$ to configure the controlled components of the EA
- 7: Execute the evolutionary search step and obtain the next population P'
- 8: Update FEs according to the newly evaluated solutions
- 9: *// Construct feedback for MARL-AC*
- 10: Construct the next population observation $\mathbf{O}'$ from P'
- 11: Obtain the next agent-specific states $\{\mathbf{s}'_k\}_{k=1}^K$ from $\mathbf{O}'$ through the deployed representation module
- 12: Compute reward r according to the search performance improvement
- 13: Store transition $\tau = (\{\mathbf{s}_k\}_{k=1}^K, \mathbf{a}, r, \{\mathbf{s}'_k\}_{k=1}^K)$ in $\mathcal{R}$
- 14: *// Update downstream policies online*
- 15: Update MARL-AC policies based on the transitions in $\mathcal{R}$
- 16: $P \leftarrow P'$
- 17: **end while**
- 18: **return** P

$D + M$ is the observation dimension. Following Algorithm 3, the deployed representation module transforms $\mathbf{O}_t$ into shared population tokens, denoted by $\mathbf{Z}_t \in \mathbb{R}^{N_z \times d_z}$, which are further mapped into agent-specific states through the trained readout heads. In this way, the same population observation is converted into multiple state views, each tailored to one decision agent in MARL-AC.

The resulting agent-specific states are then fed into the corresponding agents in MARL-AC. Specifically, based on the state representation $\mathbf{s}_{t,k}$, the k -th agent selects an action $a_{t,k}$ according to its policy $\pi_k(\cdot)$. The actions of all agents are further combined into a joint action $\mathbf{a}_t = [a_{t,1}, a_{t,2}, \dots, a_{t,K}]$, which is used to configure the controlled components of the EA at generation t . In this way, the learned representation module serves as the interface between the population observation and the downstream multi-agent decision process.

After the joint action is executed, the EA performs the corresponding search step and produces the next population, from which a new population observation can be constructed for the following generation. Meanwhile, reward feedback is collected according to the search performance improvement induced by the selected actions. Based on the collected transitions, which consist of the generated state representations, the executed actions, the received rewards, and the next agent-specific states, the multiple agents are optimized online during

TABLE I
COMPARISON OF THE ACTION, STATE, AND REWARD SETTINGS IN DIFFERENT MARL-ASSISTED EAS

Element	MADE	MAMOEAD	CMODRL	MARL-AC-Sparse
Action	Mating strategy: random, fitness-biased, or neighborhood-based (discrete); DE operator: four DE operators (discrete); Operator parameter: four F/CR settings (discrete).	Weight adjustment: Yes / No (discrete); Neighborhood size: {15, 20, 25, 30} (discrete); Reproduction operator: four DE operators (discrete); Operator parameter: {0.4, 0.5, 0.6, 0.7} (discrete).	Environment factor: [0, CV_{mean}] (continuous); Operator: DE and GA operators (discrete).	Guiding-vector: fitness-, sparsity-, random-based (discrete); Selection ratio: [0.5, 1.0] (continuous); Sparse variation operator: GLP-GA and MSKEA (discrete).
State	Compared state representation variants: Handcrafted; DAE; NeurELA; MASSR.			
Reward	Fitness-improvement reward: $r_t = \Delta f_{best} + \Delta f_{mean} + \rho_{rep}$, where ρ_{rep} denotes the successful replacement rate.	Triangle-based reward using IGD improvement; reward is given only when a new best-so-far metric value is found.	Archive-improvement reward: $r_t = \Delta c + \Delta d + \Delta f$; Δc , Δd , and Δf denote convergence, diversity, and feasibility improvements.	HV-improvement reward: $r_t = \Delta HV$, where ΔHV denotes the hypervolume change between consecutive populations.

TABLE II
TRAINING HYPERPARAMETER SETTINGS OF MASSR

Parameter	Setting
Input observation dimension	$C = D + M$
Embedding dimension	$d_h = 32$
Token dimension	$d_z = 32$
Number of selected tokens	$N_z = 12$
State dimension	$d_s = 16$ per readout head
Guidance dimension	$d_g = 12$
Guidance mask	$\mathbf{m}_k \in \{0, 1\}^{12}$
Optimizer	Adam
Learning rate	1.0×10^{-3}
Batch size	32
Training epochs	40
Contrastive temperature	$\tau = 0.2$
Guidance decay coefficient	$\lambda_\alpha = 3.0$
Random data collection phase	First 20% of MaxFEs

the evolutionary search process. During policy learning, the deployed representation module provides agent-specific state inputs for downstream agents, while the MARL-AC policies are optimized online based on the collected transitions.

By integrating MASSR into MARL-AC in this way, online policy learning can be performed on learned agent-specific states rather than directly using handcrafted state representations as policy inputs. This enables downstream agents to make decisions based on informative and specialized state inputs, while also providing a modular representation interface for different MARL-assisted EA frameworks.

IV. EMPIRICAL STUDIES

To evaluate the effectiveness of the proposed MASSR under different optimization scenarios, we embed it into four MARL-assisted EA testbeds, namely MADE for single-objective optimization, MAMOEAD [23] for multi-objective optimization [42], CMODRL [12] for constrained multi-objective optimization [43], and MARL-AC-Sparse for large-scale sparse multi-objective optimization. These instantiations cover different agent roles, action spaces, and reward definitions, thereby providing diverse evaluation scenarios for examining whether MASSR can serve as a general state representation module for MARL-AC. A detailed comparison of their action, state, and reward settings is summarized in Table I. For each MARL-assisted EA, MASSR is incorporated by replacing the original

state representation module with learned agent-specific states, while keeping the downstream policy learning procedure, action space, reward design, and evolutionary optimization process unchanged.

For each downstream algorithm, MASSR is compared with three representative state representation methods, namely handcrafted states [31], DAE-based states [12], and NeurELA-based states [6], under the same optimization and policy learning settings. The DAE-based and NeurELA-based methods are used as generic learned representation baselines, and they are not specifically designed to exploit multi-objective information or role-specific multi-agent decision requirements. Therefore, this comparison mainly examines whether the proposed MASSR can provide more suitable state representations for MARL-AC scenarios by incorporating population-level and role-related evolutionary information. The main hyperparameter settings of MASSR are summarized in Table II. In addition, the guidance indicator definitions and the head-specific guidance masks used in different MARL-AC testbeds are provided in Section SII and Table SII of the Supplementary Materials.

A. Settings of Algorithms

In the experiments, MAMOEAD and CMODRL are adopted as two representative existing MARL-assisted EAs, while MADE and MARL-AC-Sparse are implemented as additional MARL-AC testbeds to extend the evaluation to single-objective optimization and large-scale sparse multi-objective optimization, respectively. For MAMOEAD, the original MOEA/D-based decomposition framework and VDN-based cooperative policy learning scheme are retained. For CMODRL, the original three-stage framework, including initialization search, random data collection, and DRL-based decision making, is retained. MADE is implemented on a differential evolution framework as a single-objective MARL-AC testbed, where multiple agents are used to configure heterogeneous search components. MARL-AC-Sparse is implemented on a sparse evolutionary search framework as a large-scale sparse multi-objective MARL-AC testbed, where multiple agents are used to adaptively configure sparse search-related components.

For the five existing RL-assisted EAs compared in this study, the population size is uniformly set to 100. MRL-

MOEA [44] sets $p_m = 1/D$, $CR = 1$, and $F = 0.5$. AMODE-DRL [45] sets the discount factor to 0.9, with the learning rate of DDQN set to 1×10^{-2} , and the actor and critic learning rates of DDPG set to 1×10^{-4} and 1×10^{-3} , respectively. LRMODE [46] adopts four mutation strategies. MOEA-ASF [47] uses a DQN with hidden sizes 128, 256, 128, 64, and 32. RLMMDE [48] samples F and CR from Gaussian distributions and sets the discount factor to 0.8. For the compared state representation methods, the handcrafted states involve no additional trainable parameters. Specifically, for MAMOE/D, the handcrafted state follows the manually designed state indicators used in its original implementation, while for MADE, CMODRL, and MARL-AC-Sparse, a unified 12-dimensional statistical state is constructed from population-level search information for comparison. The DAE-based method follows the original autoencoder setting to learn compact population representations from evolutionary observations. The NeurELA-based method adopts the original population information embedding and two-stage attention-based encoder, where the hidden dimension is set to 16 and a single-head attention mechanism is used.

B. Settings of Problems

All experiments are implemented in MATLAB and conducted on the evolutionary multi-objective optimization platform PlatEMO v4.13 [49], [50]. Each algorithm is independently executed 30 times on each test instance. For the MADE-based methods, experiments are carried out on the BBOB test suite [51], where the maximum number of function evaluations is set to 20 000 and the population size is set to $N = 100$. For the MAMOE/D-based methods, experiments are conducted on the DTLZ and WFG test suites, where the maximum number of function evaluations is set to 63 000 and the population size is set to $N = 210$. For the CMODRL-based methods, experiments are conducted on the DOC and LIR-CMOP test suites. The maximum number of function evaluations is set to 300 000 for DOC [52] and 200 000 for LIR-CMOP [53], and the population size is set to $N = 100$. For the MARL-AC-Sparse-based methods, experiments are carried out on the SMOP test suite [54], where the number of decision variables is set to 500 and 1 000, the sparsity ratio of the Pareto-optimal solutions is set to 0.1, the maximum number of function evaluations is set to $100D$, and the population size is set to $N = 100$. For single-objective optimization, the performance is evaluated by the best fitness value obtained at the end of the search. For multi-objective optimization, the quality of the obtained solution sets is evaluated by the inverted generational distance (IGD) metric [55], [56], where 10 000 reference points are uniformly sampled from each Pareto front to calculate the IGD value. In addition, the Wilcoxon rank-sum test [57] with a significance level of 0.05 is employed to examine the statistical significance of performance differences. In the comparison, “+”, “−”, and “ $\approx$ ” indicate that the result of a compared method is significantly better than, significantly worse than, or statistically similar to that of the corresponding MASSR-embedded method, respectively.

TABLE III
MEDIAN BEST FITNESS VALUES ON BBOB_F1–BBOB_F24 OBTAINED BY MADE WITH HANDCRAFTED, DAE-BASED, NEURELA-BASED, AND PROPOSED MASSR-BASED STATE REPRESENTATIONS

Problem	D	MADE			
		Handcrafted	DAE-based	NeurELA-based	MASSR-based
BBOB_F1	30	4.7954e-3 (2.52e-3)	3.9677e-3 (2.33e-3)	3.3855e-3 (1.26e-3)	3.3599e-3 (2.20e-3)
BBOB_F2	30	8.7464e+0 (8.11e+0)	8.4260e+0 (4.48e+0)	8.3078e+0 (8.02e+0)	8.0283e+0 (6.99e+0)
BBOB_F3	30	1.7302e+2 (1.20e+1)	1.6837e+2 (1.87e+1)	1.7254e+2 (1.50e+1)	1.7286e+2 (1.27e+1)
BBOB_F4	30	2.1145e+2 (1.82e+1)	2.1242e+2 (1.29e+1)	1.9770e+2 (2.33e+1)	2.0067e+2 (1.30e+1)
BBOB_F5	30	2.3294e-1 (7.96e-2)	2.1676e-1 (4.42e-2)	2.4005e-1 (7.58e-2)	1.9599e-1 (8.27e-2)
BBOB_F6	30	1.1670e-1 (5.31e-2)	1.1230e-1 (5.86e-2)	1.1176e-1 (4.96e-2)	9.8861e-2 (5.55e-2)
BBOB_F7	30	1.0101e+0 (3.54e-1)	7.0112e-1 (3.79e-1)	6.1336e-1 (2.62e-1)	8.3887e-1 (5.44e-1)
BBOB_F8	30	3.1127e+1 (1.02e+1)	3.3501e+1 (1.35e+1)	3.2950e+1 (1.36e+1)	3.0479e+1 (7.25e+0)
BBOB_F9	30	3.4065e+1 (1.67e+1)	3.3339e+1 (1.37e+1)	3.4806e+1 (1.61e+1)	3.2301e+1 (1.27e+1)
BBOB_F10	30	5.9921e+3 (2.63e+3)	4.9978e+3 (1.98e+3)	4.6944e+3 (2.77e+3)	4.4891e+3 (1.83e+3)
BBOB_F11	30	2.2600e+1 (1.15e+1)	2.1764e+1 (6.53e+0)	1.9574e+1 (7.59e+0)	1.8693e+1 (5.37e+0)
BBOB_F12	30	4.2989e+3 (2.04e+3)	3.4345e+3 (1.78e+3)	3.1746e+3 (1.45e+3)	3.7800e+3 (2.54e+3)
BBOB_F13	30	2.7348e+1 (5.11e+0)	2.8279e+1 (6.56e+0)	2.9087e+1 (7.06e+0)	2.6527e+1 (7.35e+0)
BBOB_F14	30	1.9218e-2 (5.79e-3)	1.6246e-2 (3.75e-3)	1.4569e-2 (4.30e-3)	1.7735e-2 (6.00e-3)
BBOB_F15	30	1.9658e+2 (1.23e+1)	1.9920e+2 (1.42e+1)	1.9744e+2 (1.21e+1)	1.9836e+2 (1.15e+1)
BBOB_F16	30	2.8246e+2 (2.49e+0)	2.8856e+2 (1.90e+0)	2.7858e+2 (2.52e+0)	2.7638e+1 (2.40e+0)
BBOB_F17	30	4.4000e-1 (1.36e-1)	3.4239e-1 (1.33e-1)	4.2580e-1 (2.12e-1)	4.0967e-1 (1.20e-1)
BBOB_F18	30	2.6451e+0 (6.07e-1)	2.7661e+0 (7.53e-1)	2.5618e+0 (7.47e-1)	2.9221e+0 (1.07e+0)
BBOB_F19	30	5.6707e+0 (3.72e-1)	5.8460e+0 (2.80e-1)	5.5377e+0 (3.58e-1)	5.7501e+0 (3.09e-1)
BBOB_F20	30	2.9720e+0 (1.08e-1)	2.9548e+0 (1.20e-1)	2.9655e+0 (8.99e-2)	2.9508e+0 (3.96e-2)
BBOB_F21	30	1.6900e+0 (3.53e-1)	1.3530e+0 (5.14e-1)	1.5121e+0 (2.76e-1)	1.5932e+0 (1.47e-1)
BBOB_F22	30	3.4795e+0 (1.66e+0)	3.6240e+0 (1.87e+0)	3.6844e+0 (1.89e+0)	3.2146e+0 (1.82e+0)
BBOB_F23	30	3.2474e+0 (3.69e-1)	3.1561e+0 (3.43e-1)	3.1165e+0 (3.10e-1)	3.1672e+0 (4.50e-1)
BBOB_F24	30	2.3696e+2 (2.54e+1)	2.2773e+2 (1.95e+1)	2.3687e+2 (2.23e+1)	2.3618e+2 (1.96e+1)
+/−/≈		0/6/18	1/6/17	2/4/18	

C. Comparison of State Representation Methods across Four Optimization Scenarios

1) *Single-objective Optimization on BBOB*: Table III reports the median best fitness values obtained by different state representation methods embedded into MADE on the 30-dimensional BBOB test suite. MASSR-MADE achieves the best median results on 12 out of 24 test functions, including F1, F2, F5, F6, F8–F11, F13, F16, F20, and F22. According to the Wilcoxon rank-sum test, the handcrafted, DAE-based, and NeurELA-based states are significantly worse than MASSR on 6, 6, and 4 functions, respectively, while they are statistically similar to MASSR on most of the remaining functions. These results show that MASSR can provide competitive and often better state representations than the compared alternatives in the single-objective MADE testbed. The relatively large number of statistically similar cases is also reasonable, because multi-agent configuration is already effective for single-objective optimization, and simple scalar indicators or compact learned features can often capture sufficient population information in this scenario. MASSR is not the best-performing method on all BBOB functions. The few significant failure cases mainly occur on F19 and F21, two highly multimodal functions with complex global structures, where early population observations may not provide sufficiently stable role-specific patterns. Although the DAE-based and NeurELA-based representations were developed under different optimization settings, both remain effective in providing compact population descriptions and achieve competitive performance on several BBOB functions.

2) *Multi-objective Optimization on DTLZ and WFG*: Table IV reports the IGD results obtained by different state representation methods embedded into MAMOE/D on the DTLZ and WFG test suites. MASSR-MAMOE/D achieves strong overall performance, outperforming the handcrafted, DAE-based, and NeurELA-based states on 13, 14, and 14 out

TABLE IV
MEDIAN IGD VALUES ON DTLZ1–DTLZ7 AND WFG1–WFG9
OBTAINED BY MAMOE/D WITH HANDCRAFTED, DAE-BASED,
NEURELA-BASED, AND THE PROPOSED MASSR-BASED STATE
REPRESENTATIONS

Problem	M	D	MAMOE/D			
			Handcrafted	DAE-based	NeurELA-based	MASSR-based
DTLZ1	3	7	1.2243e+0 (1.26e+0)	7.4641e-1 (7.21e-1)	1.5428e-1 (2.04e-1)	1.4921e-2 (4.49e-4)
DTLZ2	3	12	4.0967e-2 (1.47e-3)	4.2840e-2 (1.24e-3)	4.1117e-2 (8.10e-4)	3.7392e-2 (3.52e-4)
DTLZ3	3	12	2.5936e+1 (2.04e+1)	1.8821e+1 (1.24e+1)	1.2826e+1 (9.47e+0)	2.6255e+1 (1.06e+1)
DTLZ4	3	12	5.5646e-2 (5.41e-3)	5.8149e-2 (3.78e-2)	6.7329e-2 (6.45e-2)	3.8251e-2 (4.94e-4)
DTLZ5	3	12	6.0700e-3 (1.34e-3)	5.0184e-3 (9.42e-4)	3.2994e-3 (1.76e-4)	2.1985e-3 (4.52e-5)
DTLZ6	3	12	2.6765e-3 (1.16e-4)	2.6133e-3 (1.08e-4)	2.6917e-3 (1.28e-4)	2.1175e-3 (4.65e-5)
DTLZ7	3	22	1.2889e-1 (6.33e-2)	1.0212e-1 (4.12e-2)	8.8916e-2 (5.22e-2)	4.2957e-2 (9.66e-4)
WFG1	3	12	1.1364e+0 (1.12e-1)	8.6754e-1 (9.04e-2)	7.6558e-1 (7.07e-2)	6.2114e-1 (9.44e-2)
WFG2	3	12	1.4564e-1 (6.62e-3)	1.4572e-1 (7.00e-3)	1.4165e-1 (3.96e-3)	1.2505e-1 (3.20e-3)
WFG3	3	12	1.0604e-1 (1.84e-2)	1.1547e-1 (1.17e-2)	8.7181e-2 (7.92e-3)	8.1917e-2 (6.40e-3)
WFG4	3	12	2.0460e-1 (8.61e-3)	2.0729e-1 (8.14e-3)	2.0122e-1 (6.07e-3)	1.8406e-1 (4.82e-3)
WFG5	3	12	1.6608e-1 (2.34e-3)	1.6849e-1 (2.20e-3)	1.6852e-1 (2.93e-3)	1.7219e-1 (2.59e-3)
WFG6	3	12	2.1505e-1 (5.27e-2)	2.7738e-1 (4.50e-2)	2.9607e-1 (2.94e-2)	2.2136e-1 (7.03e-2)
WFG7	3	12	1.6199e-1 (4.38e-3)	1.6173e-1 (4.03e-3)	1.5881e-1 (2.39e-3)	1.5423e-1 (3.19e-3)
WFG8	3	12	2.5414e-1 (9.78e-3)	2.6519e-1 (4.98e-3)	2.5327e-1 (3.55e-3)	2.4519e-1 (3.38e-3)
WFG9	3	12	2.2962e-1 (7.47e-2)	2.8578e-1 (5.47e-2)	2.7168e-1 (6.56e-2)	1.8832e-1 (6.71e-2)
+ / - / ≈			2/13/1	2/14/0	2/14/0	

of 16 test instances, respectively. This demonstrates the effectiveness of the learned agent-specific states for cooperative MOEA/D configuration. MASSR shows clear advantages on DTLZ7 and most WFG problems, but it is not consistently superior on every instance. The main exceptions are DTLZ3 and WFG5, where all three compared state representations significantly outperform MASSR. This may be because the population observations collected during the early random configuration phase are insufficient for learning stable role-specific patterns on these problems. To further examine whether these performance gaps are related to the limited representativeness of early-stage observations, Section SIII of the Supplementary Materials provides an additional Stage-II fine-tuning analysis. The results on DTLZ3 and WFG5 show that Stage-II fine-tuning can use later population observations to further adapt the representation module and improve the performance.

Figs. S1 and S2 of the Supplementary Materials further illustrate the search behavior of different state representations. As shown in Fig. S1 of the Supplementary Materials, MASSR-MAMOE/D obtains a better-distributed solution set on DTLZ7, covering the disconnected Pareto front segments more uniformly than the compared methods. As shown in Fig. S2, MASSR-MAMOE/D generally reaches lower IGD values on representative problems such as DTLZ7, WFG1, and WFG9, indicating better convergence behavior and final solution quality.

3) *Constrained Multi-objective Optimization on DOC and LIR-CMOP*: Table V reports the IGD results obtained by different state representation methods embedded into CMODRL on the DOC and LIR-CMOP test suites. MASSR-CMODRL achieves competitive performance on most test instances, showing that the learned agent-specific states can also be effective in constrained multi-objective optimization. This is meaningful because CMODRL involves two agents with different roles, i.e., controlling the environment factor for constraint handling and selecting offspring generation operators. By providing role-specific states, MASSR can better support the coordination among feasibility, convergence, and diversity during constrained search. Fig. S3 of the Supplemen-

TABLE V
MEDIAN IGD VALUES ON DOC1–DOC9 AND
LIR-CMOP1–LIR-CMOP14 OBTAINED BY CMODRL WITH
HANDCRAFTED, DAE-BASED, NEURELA-BASED, AND PROPOSED
MASSR-BASED STATE REPRESENTATIONS

Problem	M	D	CMODRL			
			Handcrafted	DAE-based	NeurELA-based	MASSR-based
DOC1	2	6	3.8938e-1 (2.28e-1)	3.5177e-1 (2.48e-1)	3.1402e-1 (3.02e-1)	3.1130e-1 (2.25e-1)
DOC2	2	16	7.3319e-1 (1.34e+0)	6.9899e-1 (7.01e-1)	4.8747e-1 (5.12e-2)	4.7535e-1 (1.65e-1)
DOC3	2	10	3.3725e+2 (2.59e+2)	3.2619e+2 (2.42e+2)	2.5601e+2 (2.02e+2)	2.4518e+2 (2.05e+2)
DOC4	2	8	1.8951e-1 (1.28e-1)	9.8412e-2 (9.12e-2)	1.3201e-1 (1.01e-1)	1.2171e-1 (1.12e-1)
DOC5	2	8	NaN (NaN)	NaN (NaN)	NaN (NaN)	3.7867e+1 (9.11e+1)
DOC6	2	11	4.2626e-1 (5.93e-1)	1.2693e-1 (2.45e-1)	3.5346e-1 (4.45e-1)	4.4233e-1 (5.81e-1)
DOC7	2	11	9.2430e-1 (8.29e-1)	1.1925e+0 (1.24e+0)	1.2486e+0 (1.05e+0)	9.3297e-1 (1.13e+0)
DOC8	3	10	7.4730e+1 (2.72e+1)	3.3927e+1 (3.33e+1)	6.0151e+1 (6.23e+1)	3.9575e+1 (3.28e+1)
DOC9	3	11	5.6570e-2 (3.85e-2)	4.8727e-2 (1.43e-2)	5.6944e-2 (3.97e-2)	3.9122e-2 (1.57e-2)
LIR-CMOP1	2	30	6.3984e-2 (2.92e-2)	4.5247e-2 (2.71e-2)	4.5546e-2 (2.96e-2)	4.2650e-2 (2.76e-2)
LIR-CMOP2	2	30	2.1832e-2 (7.06e-3)	2.0749e-2 (7.58e-3)	2.0266e-2 (1.07e-2)	1.5417e-2 (7.65e-3)
LIR-CMOP3	2	30	3.5749e-2 (1.71e-2)	3.0807e-2 (2.73e-2)	2.9935e-2 (1.77e-2)	1.7057e-2 (1.38e-2)
LIR-CMOP4	2	30	3.4175e-2 (2.29e-2)	3.9776e-2 (2.36e-2)	2.6003e-2 (1.78e-2)	2.1882e-2 (1.80e-2)
LIR-CMOP5	2	30	9.0311e-3 (1.45e-3)	8.0877e-3 (1.05e-3)	6.6378e-3 (5.08e-4)	6.8291e-3 (7.30e-4)
LIR-CMOP6	2	30	8.9093e-3 (9.85e-4)	8.0387e-3 (1.02e-3)	6.7025e-3 (4.80e-4)	7.1888e-3 (6.98e-4)
LIR-CMOP7	2	30	8.1859e-3 (8.03e-4)	7.7825e-3 (6.00e-4)	7.8378e-3 (8.37e-4)	7.5185e-3 (5.07e-4)
LIR-CMOP8	2	30	8.2963e-3 (1.24e-3)	7.9032e-3 (4.82e-4)	7.8383e-3 (5.42e-4)	7.5671e-3 (2.96e-4)
LIR-CMOP9	2	30	4.1439e-2 (3.31e-2)	2.9817e-2 (3.08e-2)	1.0684e-2 (1.33e-2)	1.1460e-2 (1.34e-2)
LIR-CMOP10	2	30	9.1034e-3 (9.03e-4)	8.5545e-3 (6.81e-4)	6.6991e-3 (5.21e-4)	6.9988e-3 (5.59e-4)
LIR-CMOP11	2	30	2.7116e-3 (4.20e-4)	2.4985e-3 (1.01e-4)	2.5758e-3 (2.01e-4)	2.4408e-3 (6.61e-5)
LIR-CMOP12	2	30	3.6480e-3 (6.21e-4)	3.5962e-3 (6.47e-4)	3.5142e-3 (5.21e-4)	3.3653e-3 (4.47e-4)
LIR-CMOP13	3	30	9.9853e-2 (4.61e-3)	1.0519e-1 (3.29e-3)	1.0507e-1 (3.84e-3)	9.6287e-2 (4.32e-3)
LIR-CMOP14	3	30	9.7751e-2 (1.32e-3)	9.8655e-2 (1.48e-3)	9.9000e-2 (1.45e-3)	9.7509e-2 (1.43e-3)
+ / - / ≈			0/16/7	1/14/8	2/10/11	

TABLE VI
MEDIAN IGD VALUES ON SMOP1–SMOP8 OBTAINED BY
MARL-AC-SPARSE WITH HANDCRAFTED, DAE-BASED,
NEURELA-BASED, AND PROPOSED MASSR-BASED STATE
REPRESENTATIONS

Problem	M	D	MARL-AC-Sparse			
			Handcrafted	DAE-based	NeurELA-based	MASSR-based
SMOP1	2	500	2.9788e-3 (6.41e-4)	3.0912e-3 (8.12e-4)	2.9601e-3 (1.01e-3)	2.7742e-3 (8.45e-4)
		1000	3.0781e-3 (1.19e-3)	2.9578e-3 (9.26e-4)	3.2006e-3 (1.34e-3)	2.5793e-3 (3.54e-4)
SMOP2	2	500	4.6665e-3 (2.78e-3)	4.5089e-3 (3.40e-3)	4.4098e-3 (2.57e-3)	3.9196e-3 (2.15e-3)
		1000	4.3847e-3 (2.71e-3)	3.6393e-3 (1.73e-3)	4.2535e-3 (1.80e-3)	3.0700e-3 (8.18e-4)
SMOP3	2	500	2.7093e-3 (5.34e-4)	2.6141e-3 (4.56e-4)	2.5263e-3 (3.58e-4)	2.5045e-3 (3.65e-4)
		1000	2.6722e-3 (3.70e-4)	2.4852e-3 (2.76e-4)	2.6118e-3 (3.87e-4)	2.4085e-3 (3.09e-4)
SMOP4	2	500	2.4483e-3 (2.45e-4)	2.3975e-3 (2.56e-4)	2.4610e-3 (2.14e-4)	2.4337e-3 (2.22e-4)
		1000	2.5432e-3 (2.56e-4)	2.5123e-3 (2.29e-4)	2.3711e-3 (2.14e-4)	2.3146e-3 (2.02e-4)
SMOP5	2	500	3.7330e-3 (3.09e-4)	3.6768e-3 (2.57e-4)	3.6642e-3 (3.27e-4)	3.7335e-3 (3.52e-4)
		1000	3.0474e-3 (2.21e-4)	2.9604e-3 (2.35e-4)	3.0490e-3 (2.50e-4)	2.9802e-3 (2.26e-4)
SMOP6	2	500	4.2619e-3 (4.07e-4)	4.3101e-3 (5.10e-4)	4.2498e-3 (5.47e-4)	4.2165e-3 (5.58e-4)
		1000	3.3831e-3 (4.13e-4)	3.4399e-3 (3.91e-4)	3.4380e-3 (3.69e-4)	3.5231e-3 (4.71e-4)
SMOP7	2	500	3.4252e-3 (1.26e-3)	3.9682e-3 (1.42e-3)	3.5358e-3 (8.11e-4)	3.7610e-3 (1.25e-3)
		1000	2.9362e-3 (4.69e-4)	3.0761e-3 (9.23e-4)	2.7147e-3 (3.43e-4)	3.0412e-3 (6.94e-4)
SMOP8	2	500	1.4018e-1 (8.49e-3)	1.4060e-1 (1.18e-2)	1.4053e-1 (1.23e-2)	1.3621e-1 (1.14e-2)
		1000	1.6879e-1 (1.08e-1)	1.4986e-1 (8.80e-3)	1.4926e-1 (6.40e-3)	1.3842e-1 (8.67e-3)
+ / - / ≈			0/7/9	0/6/10	1/6/9	

tary Materials further presents the objective-space distributions obtained by different methods on DOC4 and DOC5. MASSR-CMODRL generates solutions that are closer to the Pareto front and exhibit a more reasonable spread than the compared methods, indicating better balance between convergence and diversity under complex constraints.

4) *Large-scale Sparse Multi-objective Optimization on SMOP*: Table VI reports the IGD results obtained by different state representation methods embedded into MARL-AC-Sparse on SMOP1–SMOP8 with 500 and 1000 decision variables. MASSR-MARL-AC-Sparse achieves the best median IGD values on 10 out of 16 test instances. According to the Wilcoxon rank-sum test, the handcrafted, DAE-based, and NeurELA-based states are significantly worse than MASSR on 7, 6, and 6 instances, respectively, while only the NeurELA-based state significantly outperforms MASSR on one instance. These results indicate that MASSR can also provide effective state representations for large-scale sparse multi-objective optimization. An additional performance and

runtime analysis of Stage-II fine-tuning is provided in Section SIII of the Supplementary Materials.

D. Comparison with Existing RL-assisted EAs

To further examine the algorithm-level competitiveness of MASSR, we compare MASSR-MAMOE/D with five representative RL-assisted EAs, including MRL-MOEA, AMODE-DRL, LRMODE, MOEA-ASF, and RLMMDE. The detailed IGD results and the representative population-distribution visualization are provided in Section SIV of the Supplementary Materials. Overall, MASSR-MAMOE/D achieves competitive performance on the DTLZ and WFG test suites. This suggests that the learned agent-specific states can be effectively embedded into an existing RL-assisted EA framework and can support the coordination of multiple configurable components during evolutionary search.

E. Effectiveness Analysis of MASSR

1) *Ablation Studies*: To examine the effects of the main components in MASSR, we construct three ablation variants based on MASSR-MAMOE/D. The first variant replaces the multi-head readout with a unified state encoder, so that all agents share the same state representation. The second variant removes the contrastive learning objective and retains only the auxiliary guidance objective, while the third variant removes the auxiliary guidance objective and retains only the contrastive learning objective. The detailed IGD results and further analysis are provided in Section SV of the Supplementary Materials. Overall, the ablation results show that removing these components tends to degrade the performance of MASSR-MAMOE/D on the tested instances.

2) *Head-wise Representation Analysis*: To further examine whether MASSR learns differentiated agent-specific states, we conduct head-wise representation analyses, with detailed variant comparisons, cosine-similarity results, and guide-loss curves provided in Section SVI of the Supplementary Materials. The supplementary results show that the complete MASSR-MAMOE/D outperforms the head-wise variants, indicating that conventional multi-head contrastive learning, identical guidance, shared states, averaged head outputs, or parameter-shared heads cannot fully reproduce the proposed design. In addition, the low pairwise cosine similarities among different heads suggest that the learned states do not collapse to identical representations, while the rapidly decreased guide prediction loss shows that the head-specific states become predictive of their role-related auxiliary guidance targets.

3) *Transferability and Fine-tuning Analysis*: The transferability results in Section SVII of the Supplementary Materials show that MASSR can be used under both online and transfer settings. The Transfer-finetune protocol, where the source-trained representation module is adapted using early observations from unseen target problems, generally achieves better performance than directly transferring the representation without fine-tuning. At the same time, the No-transfer online setting remains competitive on several instances, showing that instance-specific online learning is still a reliable and convenient default choice. These results suggest that MASSR

provides a flexible representation learning framework that can support both per-instance adaptation and cross-instance transfer across related optimization problems. To further examine zero-shot generalization beyond the MAMOE/D testbed, we additionally evaluate MASSR on MADE, CMODRL, and MARL-AC-Sparse under dimension-compatible source and target settings, covering single-objective, constrained multi-objective, and large-scale sparse multi-objective optimization. The detailed results are provided in Section SVIII and Table SIX of the Supplementary Materials.

V. CONCLUSION

This paper proposes a multi-agent self-supervised state representation framework (MASSR) with auxiliary guidance for reinforcement learning-assisted evolutionary algorithms. MASSR learns shared population tokens from population observations and derives agent-specific states through a multi-head readout mechanism, enabling heterogeneous agents to obtain differentiated decision information from the same evolving population. By jointly using contrastive self-supervision and auxiliary guidance, MASSR captures both population-level structures and role-related evolutionary information, and can be deployed as a modular state representation component for downstream policy learning. The proposed framework is instantiated in four representative RL-assisted EA testbeds and evaluated across different optimization scenarios. Experimental results show that MASSR generally provides more effective states than handcrafted, DAE-based, and NeurELA-based alternatives, while ablation, head-wise, and transfer analyses further verify its effectiveness, role-specific representation ability, and flexible applicability.

The current transfer experiments are restricted to dimension-compatible source and target problems, with the agent roles, action space, reward definition, and evolutionary framework unchanged within each testbed. Although the extended zero-shot experiments demonstrate the reuse potential of MASSR across several optimization scenarios, they do not yet establish fully heterogeneous transferability. Future work will investigate dimension-invariant representations or dimensional-alignment mechanisms to support cross-dimensional transfer. We will also explore more general state representation modules that can adapt to different agent structures, action spaces, and evolutionary frameworks. MASSR will be further evaluated on broader large-scale [58]–[60], many-objective [61], and real-world optimization problems [62]. More advanced representation models [63], [64] and adaptive updating strategies [65] will also be explored to improve its generality and adaptability.

REFERENCES

- [1] Y. Song, Y. Wu, Y. Guo, R. Yan, P. N. Suganthan, Y. Zhang, W. Pedrycz, S. Das, R. Mallipeddi, O. S. Ajani, *et al.*, “Reinforcement learning-assisted evolutionary algorithm: A survey and research opportunities,” *Swarm Evol. Comput.*, vol. 86, p. 101517, 2024.
- [2] X. Liu, J. Sun, Q. Zhang, Z. Wang, and Z. Xu, “Learning to learn evolutionary algorithm: A learnable differential evolution,” *IEEE Trans. Emerg. Topics Comput. Intell.*, vol. 7, no. 6, pp. 1605–1620, 2023.
- [3] E. Schede, J. Brandt, A. Tornede, M. Wever, V. Bengs, E. Hüllermeier, and K. Tierney, “A survey of methods for automated algorithm configuration,” *J. Artif. Intell. Res.*, vol. 75, pp. 425–487, 2022.

- [4] T. Stützle, M. López-Ibáñez, and L. Pérez-Cáceres, "Automated algorithm configuration and design," in *Proc. Genet. Evol. Comput. Conf. Companion*, 2022, pp. 997–1019.
- [5] A. G. Barto, R. S. Sutton, and C. J. C. H. Watkins, *Learning and Sequential Decision Making*, vol. 89. Amherst, MA, USA: Univ. Massachusetts Amherst, 1989.
- [6] Z. Ma, J. Chen, H. Guo, and Y.-J. Gong, "Neural exploratory landscape analysis for meta-black-box-optimization," in *Proc. Int. Conf. Learn. Represent.*, 2025, pp. 15373–15393.
- [7] H. Guo, S. Ma, Z. Huang, Y. Hu, Z. Ma, X. Zhang, and Y.-J. Gong, "Reinforcement learning-based self-adaptive differential evolution through automated landscape feature learning," in *Proc. Genet. Evol. Comput. Conf.*, 2025, pp. 1117–1126.
- [8] Y.-T. Zhong, T. Huang, X. Xiao, and Y.-J. Gong, "TRACE: A generalizable drift detector for streaming data-driven optimization," in *Proc. AAAI Conf. Artif. Intell.*, vol. 40, no. 34, 2026, pp. 28910–28918, doi: 10.1609/aaai.v40i34.40126.
- [9] Y. Zhang, Z. Wang, L. Zou, H. Dong, and X. Yi, "Neural-network-based secure state estimation under energy-constrained denial-of-service attacks: An encoding-decoding scheme," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 4, pp. 2002–2015, 2023.
- [10] Z. Li and S. Li, "Neural network model-based control for manipulator: An autoencoder perspective," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 6, pp. 2854–2868, 2021.
- [11] W. Liu, Z. Wang, X. Liu, N. Zeng, Y. Liu, and F. E. Alsaadi, "A survey of deep neural network architectures and their applications," *Neurocomputing*, vol. 234, pp. 11–26, 2017.
- [12] F. Ming, W. Gong, B. Xue, M. Zhang, and Y. Jin, "Automated configuration of evolutionary algorithms via deep reinforcement learning for constrained multiobjective optimization," *IEEE Trans. Cybern.*, vol. 55, no. 12, pp. 5877–5890, 2025.
- [13] O. Preuß, J. Rook, J. Bossek, and H. Trautmann, "MO-ELA: Rigorously expanding exploratory landscape features for automated algorithm selection in continuous multi-objective optimisation," *arXiv preprint arXiv:2602.00098*, 2026. [Online]. Available: <https://arxiv.org/abs/2602.00098>
- [14] B. van Stein, F. X. Long, M. Frenzel, P. Krause, M. Gitterle, and T. Bäck, "Doe2vec: Deep-learning based features for exploratory landscape analysis," in *Proc. Genet. Evol. Comput. Conf. Companion*, 2023, pp. 515–518.
- [15] M. V. Seiler, P. Kerschke, and H. Trautmann, "Deep-ELA: Deep exploratory landscape analysis with self-supervised pretrained transformers for single- and multiobjective continuous optimization problems," *Evol. Comput.*, vol. 33, no. 4, pp. 513–540, 2025.
- [16] M. Laskin, A. Srinivas, and P. Abbeel, "Curl: Contrastive unsupervised representations for reinforcement learning," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 5639–5650.
- [17] T.-Y. Chen, W.-N. Chen, F.-F. Wei, X.-Q. Guo, W.-X. Song, R. Zhu, Q. Lin, and J. Zhang, "The confluence of evolutionary computation and multi-agent systems: A survey," *IEEE/CAA J. Autom. Sin.*, vol. 12, no. 11, pp. 2175–2193, 2025.
- [18] L. Chen, K. Xue, L. Yuan, Y. Wang, Y. Wang, F. Sheng, and C. Qian, "Sequential multi-agent dynamic algorithm configuration," in *Proc. Adv. Neural Inf. Process. Syst.*, San Diego, CA, USA, 2025.
- [19] N. Botteghi, M. Poel, and C. Brune, "Unsupervised representation learning in deep reinforcement learning: A review," *IEEE Control Syst.*, vol. 45, no. 2, pp. 26–68, 2025.
- [20] S. Adriaensen, A. Biedenkapp, G. Shala, N. Awad, T. Eimer, M. Lindauer, and F. Hutter, "Automated dynamic algorithm configuration," *J. Artif. Intell. Res.*, vol. 75, pp. 1633–1699, 2022.
- [21] S. Liu, K. Tang, Y. Lei, and X. Yao, "On performance estimation in automatic algorithm configuration," in *Proc. AAAI Conf. Artif. Intell.*, vol. 34, no. 3, 2020, pp. 2384–2391.
- [22] F. Hutter, H. H. Hoos, K. Leyton-Brown, and T. Stützle, "ParamILS: An automatic algorithm configuration framework," *J. Artif. Intell. Res.*, vol. 36, pp. 267–306, 2009.
- [23] K. Xue, J. Xu, L. Yuan, M. Li, C. Qian, Z. Zhang, and Y. Yu, "Multi-agent dynamic algorithm configuration," *Adv. Neural Inf. Process. Syst.*, vol. 35, pp. 20147–20161, 2022.
- [24] J. Li, Y. Liu, J. Zou, S. Liu, S. Yang, and J. Zheng, "Multi-agent cooperation-based bi-criteria evolutionary many-objective optimization," *Appl. Soft Comput.*, p. 113865, 2025.
- [25] B. Qiu, K. Gao, H. Yu, and A. Sadollah, "Modelling and scheduling distributed assembly permutation flow-shops using reinforcement learning-based evolutionary algorithms," *Eng. Appl. Artif. Intell.*, vol. 142, p. 109851, 2025.
- [26] F. Ming, W. Gong, L. Wang, and Y. Jin, "Constrained multi-objective optimization with deep reinforcement learning assisted operator selection," *IEEE/CAA J. Autom. Sinica*, vol. 11, no. 4, pp. 919–931, 2024.
- [27] Y. Tian, X. Li, H. Ma, X. Zhang, K. C. Tan, and Y. Jin, "Deep reinforcement learning based adaptive operator selection for evolutionary multi-objective optimization," *IEEE Trans. Emerg. Top. Comput. Intell.*, vol. 7, no. 4, pp. 1051–1064, 2023.
- [28] H. Guo, W. Qiu, Z. Ma, X. Zhang, J. Zhang, and Y.-J. Gong, "Advancing CMA-ES with learning-based cooperative coevolution for scalable optimization," *arXiv preprint arXiv:2504.17578*, 2025. [Online]. Available: <https://arxiv.org/abs/2504.17578>
- [29] X. Liu, T. Wang, Z. Zeng, Y. Tian, and J. Tong, "Three stage based reinforcement learning for combining multiple metaheuristic algorithms," *Swarm Evol. Comput.*, vol. 95, p. 101935, 2025.
- [30] D. Li, K. Gao, P. Duan, P. N. Suganthan, and N. Wu, "Reinforcement learning assisting artificial bee colony algorithm for scheduling distributed assembly flowshops with batch delivery," *IEEE Trans. Syst., Man, Cybern., Syst.*, 2025.
- [31] Z. Ma, J. Chen, H. Guo, Y. Ma, and Y.-J. Gong, "Auto-configuring exploration-exploitation tradeoff in evolutionary computation via deep reinforcement learning," in *Proc. Genet. Evol. Comput. Conf.*, 2024, pp. 1497–1505.
- [32] H. Guo, Y. Ma, Z. Ma, J. Chen, X. Zhang, Z. Cao, J. Zhang, and Y.-J. Gong, "Deep reinforcement learning for dynamic algorithm selection: A proof-of-principle study on differential evolution," *IEEE Trans. Syst., Man, Cybern., Syst.*, vol. 54, no. 7, pp. 4247–4259, 2024.
- [33] L. Annabi and M. Garcia Ortiz, "State representation learning with recurrent capsule networks," *arXiv preprint arXiv:1812.11202*, 2018.
- [34] Z. Ma, H. Guo, J. Chen, Z. Li, G. Peng, Y.-J. Gong, Y. Ma, and Z. Cao, "MetaBox: A benchmark platform for meta-black-box optimization with reinforcement learning," in *Adv. Neural Inf. Process. Syst.*, vol. 36, 2023, pp. 10775–10795.
- [35] Z. Ma, Y.-J. Gong, H. Guo, W. Qiu, S. Ma, H. Lian, J. Zhan, K. Chen, C. Wang, Z. Huang, Z. Huang, G. Peng, R. Cheng, and Y. Ma, "MetaBox-v2: A unified benchmark platform for meta-black-box optimization," in *Adv. Neural Inf. Process. Syst.*, vol. 38, 2025.
- [36] H. Guo, Z. Ma, Y. Ma, X. Zhang, W.-N. Chen, and Y.-J. Gong, "DesignX: Human-competitive algorithm designer for black-box optimization," in *Adv. Neural Inf. Process. Syst.*, vol. 38, 2025, pp. 6582–6615.
- [37] Y. Wu, H. Guo, T. Huang, Y.-J. Gong, and Z. Ma, "Surrogate ensemble in expensive multi-objective optimization via deep Q-learning," *arXiv preprint arXiv:2602.00540*, 2026.
- [38] T. De Bruin, J. Kober, K. Tuyls, and R. Babuska, "Integrating state representation learning into deep reinforcement learning," *IEEE Robot. Autom. Lett.*, vol. 3, no. 3, pp. 1394–1401, 2018.
- [39] F. Liu, R. Tang, X. Li, W. Zhang, Y. Ye, H. Chen, H. Guo, Y. Zhang, and X. He, "State representation modeling for deep reinforcement learning based recommendation," *Knowl.-Based Syst.*, vol. 205, p. 106170, 2020.
- [40] T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, "A simple framework for contrastive learning of visual representations," in *Proc. Int. Conf. Mach. Learn.*, 2020, pp. 1597–1607.
- [41] Y. You, T. Chen, Y. Sui, T. Chen, Z. Wang, and Y. Shen, "Graph contrastive learning with augmentations," in *Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 5812–5823.
- [42] Y. Tian, X. Qi, S. Yang, C. He, K. C. Tan, Y. Jin, and X. Zhang, "A universal framework for automatically generating single- and multi-objective evolutionary algorithms," *IEEE Trans. Evol. Comput.*, 2025.
- [43] J. Liang, H. Lin, C. Yue, X. Ban, and K. Yu, "Evolutionary constrained multi-objective optimization: A review," *Vicinityearth*, vol. 1, p. 5, 2024.
- [44] J. Wang, Y. Zheng, Z. Zhang, H. Peng, and H. Wang, "A novel multi-state reinforcement learning-based multi-objective evolutionary algorithm," *Inf. Sci.*, vol. 688, Art. no. 121397, 2025.
- [45] T. Li, Y. Meng, and L. Tang, "Scheduling of continuous annealing with a multi-objective differential evolution algorithm based on deep reinforcement learning," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 2, pp. 1767–1780, 2023.
- [46] Y. Huang, W. Li, F. Tian, and X. Meng, "A fitness landscape ruggedness multiobjective differential evolution algorithm with a reinforcement learning strategy," *Appl. Soft Comput.*, vol. 96, Art. no. 106693, 2020.
- [47] Y. Tian, L. Yao, S. Shao, Y. Zhang, and X. Zhang, "Deep reinforcement learning based adaptive environmental selection for evolutionary multi-objective optimization," in *Proc. IEEE Congr. Evol. Comput. (CEC)*, 2024, pp. 1–8.
- [48] Y. Han, H. Peng, C. Mei, L. Cao, C. Deng, H. Wang, and Z. Wu, "Multi-strategy multi-objective differential evolutionary algorithm with

- reinforcement learning,” *Knowl.-Based Syst.*, vol. 277, Art. no. 110801, 2023.
- [49] Y. Tian, R. Cheng, X. Zhang, and Y. Jin, “PlatEMO: A MATLAB platform for evolutionary multi-objective optimization [educational forum],” *IEEE Comput. Intell. Mag.*, vol. 12, no. 4, pp. 73–87, 2017.
- [50] Y. Tian, W. Zhu, X. Zhang, and Y. Jin, “A practical tutorial on solving optimization problems via PlatEMO,” *Neurocomputing*, vol. 518, pp. 190–205, 2023.
- [51] N. Hansen, S. Finck, R. Ros, and A. Auger, “Real-parameter black-box optimization benchmarking 2009: Noiseless functions definitions,” INRIA, Rocquencourt, France, Tech. Rep. RR-6829, 2009.
- [52] Z.-Z. Liu and Y. Wang, “Handling constrained multiobjective optimization problems with constraints in both the decision and objective spaces,” *IEEE Trans. Evol. Comput.*, vol. 23, no. 5, pp. 870–884, 2019.
- [53] Z. Fan, W. Li, X. Cai, H. Huang, Y. Fang, Y. You, J. Mo, C. Wei, and E. Goodman, “An improved epsilon constraint-handling method in MOEA/D for CMOPs with large infeasible regions,” *Soft Comput.*, vol. 23, no. 23, pp. 12491–12510, 2019.
- [54] Y. Tian, X. Zhang, C. Wang, and Y. Jin, “An evolutionary algorithm for large-scale sparse multiobjective optimization problems,” *IEEE Trans. Evol. Comput.*, vol. 24, no. 2, pp. 380–393, Apr. 2020.
- [55] P. A. N. Bosman and D. Thierens, “The balance between proximity and diversity in multiobjective evolutionary algorithms,” *IEEE Trans. Evol. Comput.*, vol. 7, no. 2, pp. 174–188, 2003.
- [56] H. Ishibuchi, H. Masuda, Y. Tanigaki, and Y. Nojima, “Modified distance calculation in generational distance and inverted generational distance,” in *Proc. Int. Conf. Evol. Multi-Criterion Optim.*, 2015, pp. 110–125.
- [57] J. Derrac, S. García, D. Molina, and F. Herrera, “A practical tutorial on the use of nonparametric statistical tests as a methodology for comparing evolutionary and swarm intelligence algorithms,” *Swarm Evol. Comput.*, vol. 1, no. 1, pp. 3–18, 2011.
- [58] W. Qiu, Z. Wang, H. Fang, Z. Ma, and Y.-J. Gong, “A learning-based cooperative coevolution framework for heterogeneous large-scale global optimization,” *arXiv preprint arXiv:2604.01241*, 2026. [Online]. Available: <https://arxiv.org/abs/2604.01241>
- [59] S. Shao, Y. Tian, Y. Zhang, and X. Zhang, “Knowledge learning-based dimensionality reduction for solving large-scale sparse multiobjective optimization problems,” *IEEE Trans. Cybern.*, 2025.
- [60] S. Shao, Y. Tian, Y. Zhang, S. Yang, P. Zhang, C. He, X. Zhang, and Y. Jin, “Evolutionary computation for sparse multi-objective optimization: A survey,” *ACM Comput. Surv.*, vol. 57, no. 11, pp. 1–35, 2025.
- [61] H. Ishibuchi, N. Tsukamoto, and Y. Nojima, “Evolutionary many-objective optimization: A short review,” in *Proc. IEEE Congr. Evol. Comput.*, 2008, pp. 2419–2426.
- [62] T. Back, M. Emmerich, and O. M. Shir, “Evolutionary algorithms for real world applications [application notes],” *IEEE Comput. Intell. Mag.*, vol. 3, no. 1, pp. 64–67, 2008.
- [63] Z. Xu, Y. Zhang, W. Bao, H. Wang, M. Chen, H. Ye, W. Jiang, H. Yan, and J. Wang, “AutoEP: LLMs-driven automation of hyperparameter evolution for metaheuristic algorithms,” in *Proc. Int. Conf. Learn. Represent. (ICLR)*, 2026.
- [64] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” *Adv. Neural Inf. Process. Syst.*, vol. 30, 2017.
- [65] X. Sun, F. Sun, Z. Zhang, P. Li, and S. Wang, “Adaptive self-supervised learning for sequential recommendation,” *Neural Netw.*, vol. 179, p. 106570, 2024.



Xiaotong Liu received the B.Sc. degree from Anhui University of Science and Technology, Huainan, China, in 2022, and the M.Eng. degree from Wuyi University, Jiangmen, China, in 2025. He is currently pursuing the Ph.D. degree with the School of Computer Science and Technology, Anhui University, Hefei, China.

His current research interests include automated algorithm design, evolutionary computation, and reinforcement learning.



Ye Tian (Senior Member, IEEE) received the B.Sc., M.Sc., and Ph.D. degrees from Anhui University, Hefei, China, in 2012, 2015, and 2018, respectively.

He is currently a Professor with the School of Computer Science and Technology, Anhui University, Hefei, China. His current research interests include evolutionary computation and its applications. He is the recipient of the 2018, 2021, and 2024 IEEE Transactions on Evolutionary Computation Outstanding Paper Award, the 2020 IEEE Computational Intelligence Magazine Outstanding Paper Award, and the 2022 IEEE Computational Intelligence Society Outstanding PhD Dissertation Award.



Shangshang Yang received the B.Sc. and Ph.D. degrees from Anhui University, Hefei, China, in 2017 and 2022, respectively. He was a Visiting Ph.D. student at Bielefeld University, Germany in 2022. He is currently a Lecture with the School of Computer Science and Technology, Anhui University, Hefei, China.

His current research interests include evolutionary multi-objective optimization, neural architecture search, intelligent education, and graph learning. He is the recipient of the 2023 and 2026 International Conference on Data-driven Optimization of Complex Systems Best Paper Award. He got the Postdoctoral Fellowship Program (Grade B) of China Postdoctoral Science Foundation.



Zimo Sheng completed his undergraduate, master's, and doctoral studies at the University of Electronic Science and Technology of China, Chengdu, China, in 2018, 2020, and 2024, respectively.

He is currently a Lecturer with the School of Computer Science and Technology, Anhui University, Hefei, China. His research includes algorithm design and evolutionary optimization.



Xingyi Zhang (Fellow, IEEE) received the B.Sc. degree from Fuyang Normal College, Fuyang, China, in 2003, and the M.Sc. and Ph.D. degrees from Huazhong University of Science and Technology, Wuhan, China, in 2006 and 2009, respectively.

He is currently a Professor with the School of Computer Science and Technology, Anhui University, Hefei, China. His current research interests include unconventional models and algorithms of computation, evolutionary multi-objective optimization, and logistic scheduling. He is the recipient of the 2018, 2021, and 2024 IEEE Transactions on Evolutionary Computation Outstanding Paper Award and the 2020 IEEE Computational Intelligence Magazine Outstanding Paper Award.